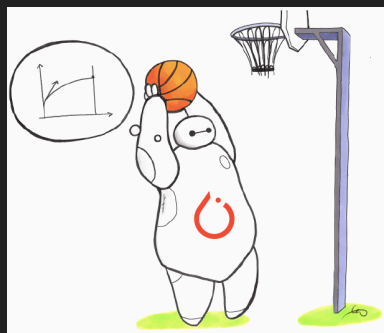
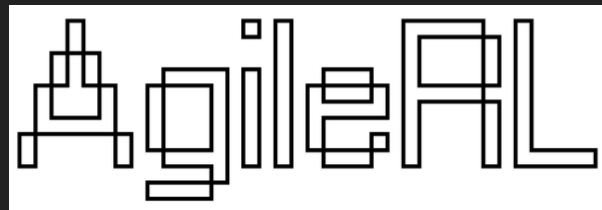


6

BIBLIOTECAS PARA APRENDIZAJE POR REFUERZO

¿QUÉ BIBLIOTECAS MÁS RELEVANTES EXISTEN PARA RL?

Agentes



Entornos



MuJoCo

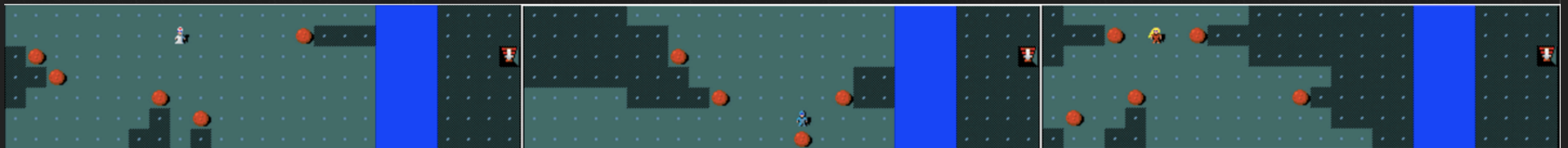


ENTORNOS



MINIHACK

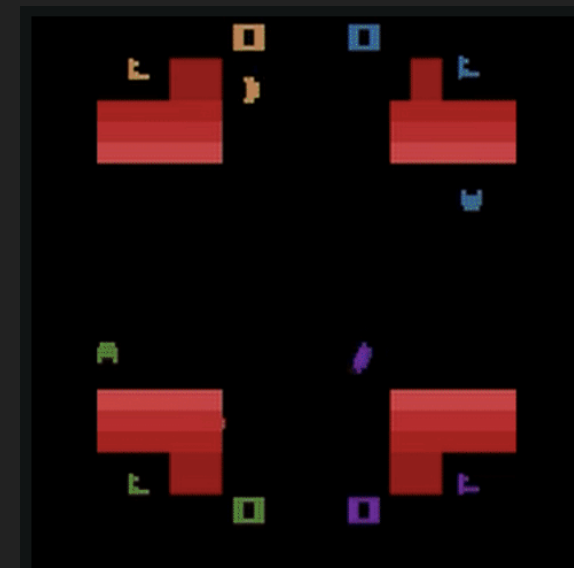
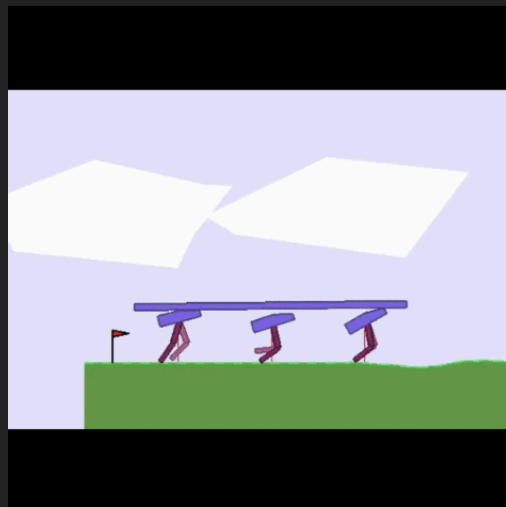
- ▶ **Propósito:** Crear entornos de RL basados en el juego NetHack, un roguelike complejo.
- ▶ **Uso principal:** Investigación académica en entornos desafiantes y con exploración profunda.
- ▶ **Características clave:**
 - ✓ Basado en NetHack Learning Environment (NLE).
 - ✓ Requiere razonamiento complejo, memoria y planificación.
 - ✓ Muy útil para probar agentes en tareas similares a juegos de texto o juegos de aventuras.
- ▶ Es una colección de implementaciones minimalistas de algoritmos de RL.
- ▶ Cada archivo .py es autocontenido (solo unas ~200 líneas).
- ▶ <https://minihack.readthedocs.io/en/latest/index.html>



PETTINGZOO



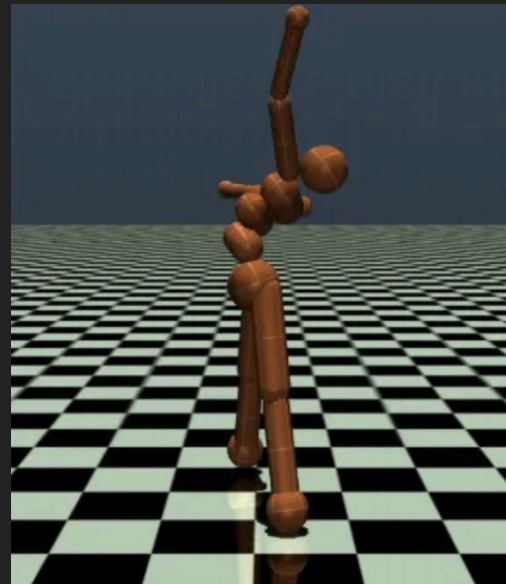
- ▶ **Propósito:** Proporcionar entornos para Multi-Agent Reinforcement Learning (MARL).
- ▶ **Uso principal:** Investigación y entrenamiento en escenarios con múltiples agentes.
- ▶ **Características clave:**
 - ✓ API estándar similar a Gymnasium pero adaptada a multi-agente.
 - ✓ Compatible con Stable-Baselines, RLlib, y otros frameworks.
 - ✓ Soporta entornos paralelos y secuenciales.
- ▶ <https://pettingzoo.farama.org/>



MUJOCO

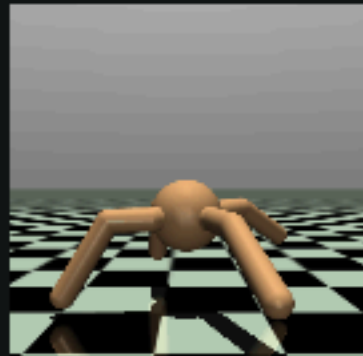
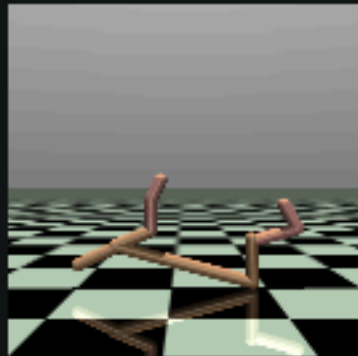
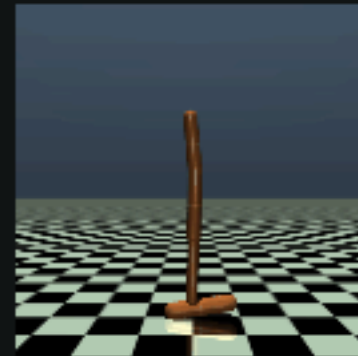
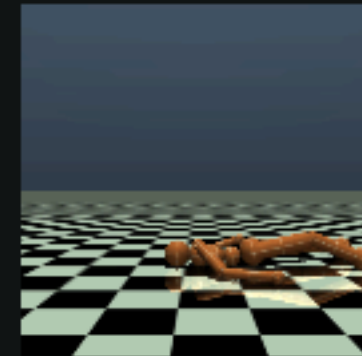
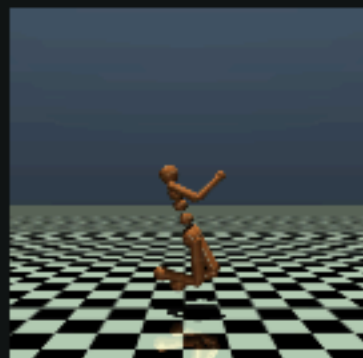
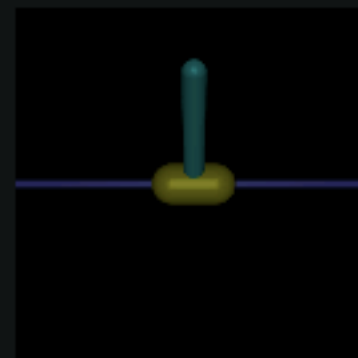
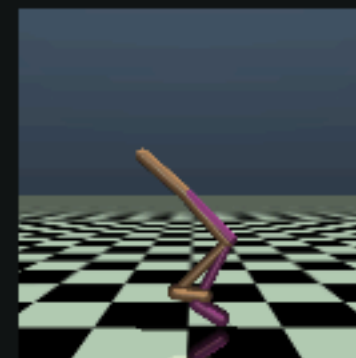
MuJoCo

- ▶ **Propósito:** Simulación física de robots y cuerpos rígidos. (Multi-Joint dynamics with Contact)
- ▶ **Uso principal:** Aprendizaje de control en robótica.
- ▶ **Características clave:**
 - ✓ Precisión física muy alta.
 - ✓ Usado como backend en varios entornos de control como los de Gymnasium.
 - ✓ Propiedad de DeepMind (Google).
- ▶ <https://mujoco.readthedocs.io/en/stable/overview.html>



MUJOCO

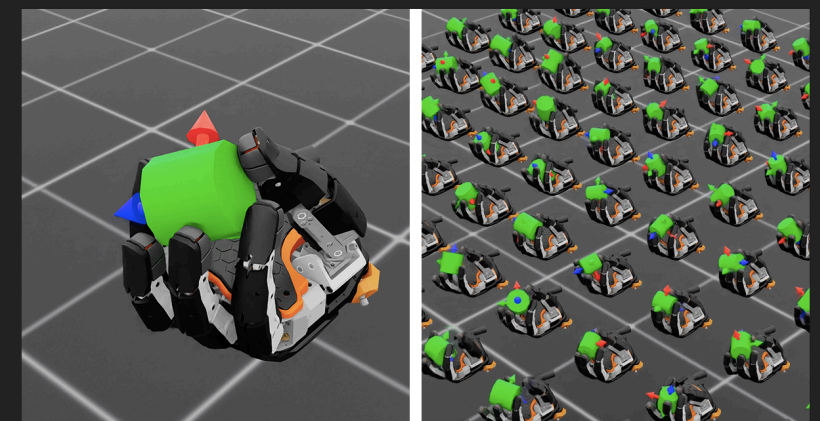
MuJoCo

AntHalf CheetahHopperHumanoid StandupHumanoidInverted Double
PendulumInverted PendulumPusherReacherSwimmerWalker2D

ISAAC LAB



- ▶ **Propósito:** Simulación física de robots.
- ▶ **Uso principal:** Entrenar robots mediante RL e Imitation Learning (IL) en entornos simulados paralelizados. Ejecuta miles de instancias de entrenamiento simultáneas sobre hardware GPU.
- ▶ **Características clave:**
 - ✓ Permite intercambiar motores de física (PhysX, Newton, MuJoCo).
 - ✓ Permite intercambiar bibliotecas de RL (RSL-RL, RL-Games, SKRL, Stable Baselines).
- ▶ <https://github.com/isaac-sim/IsaacLab>



GYMNASIUM



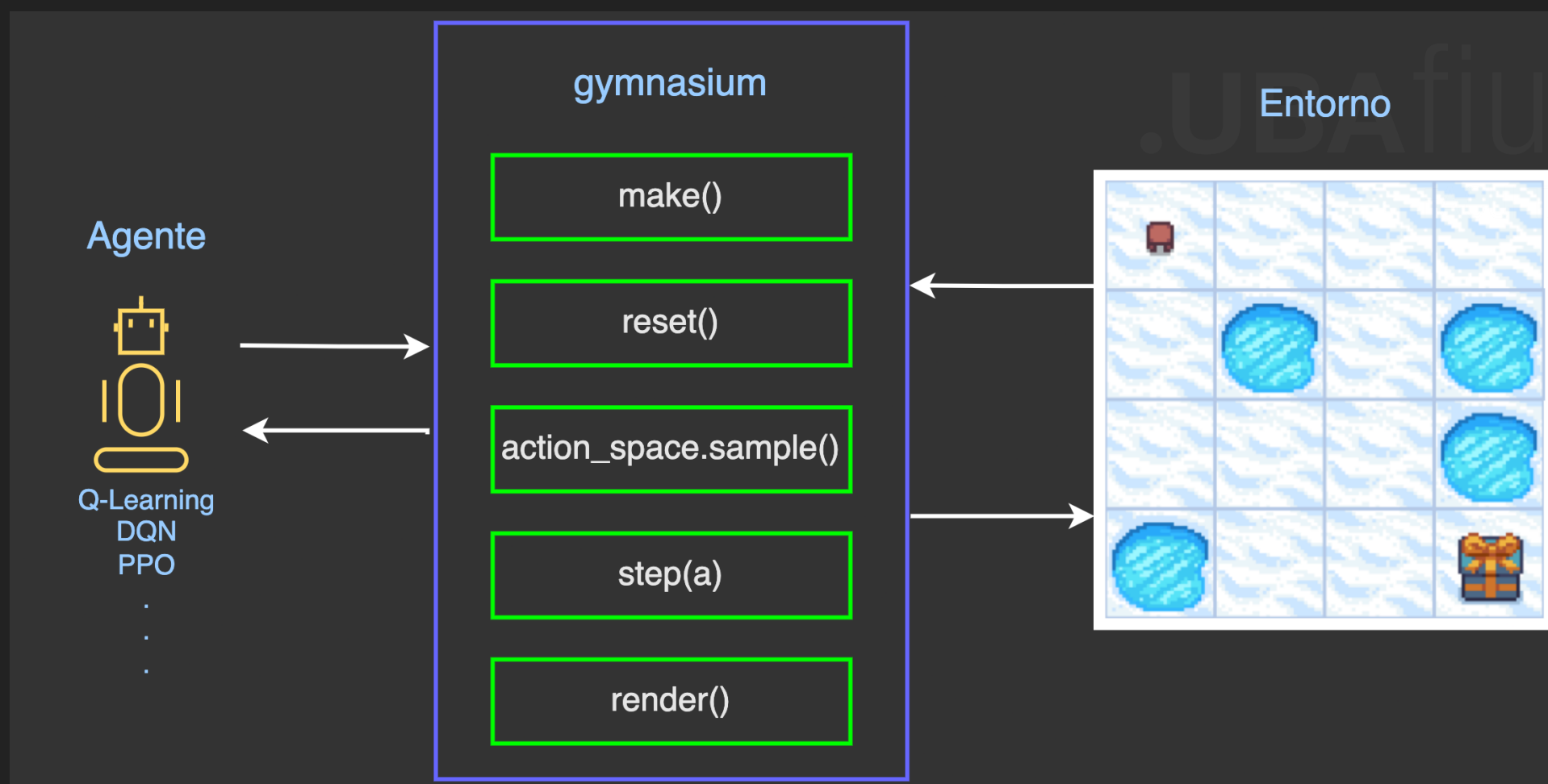
- ▶ **Propósito:** Crear y usar entornos de RL estandarizados.
- ▶ **Uso principal:** Entrenamiento y prueba de algoritmos de RL.
- ▶ **Características clave:**
 - ✓ Es el sucesor de OpenAI Gym, mantenido por Farama Foundation.
 - ✓ Interfaz simple y unificada.
 - ✓ Compatible con otras bibliotecas como Stable-Baselines y CleanRL.
- ▶ <https://gymnasium.farama.org/>



COMO USAR GYMNASIUM?



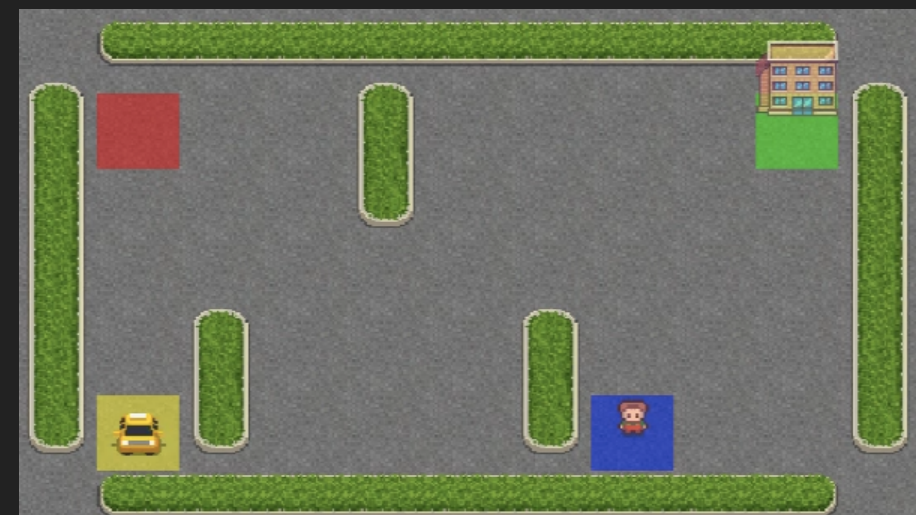
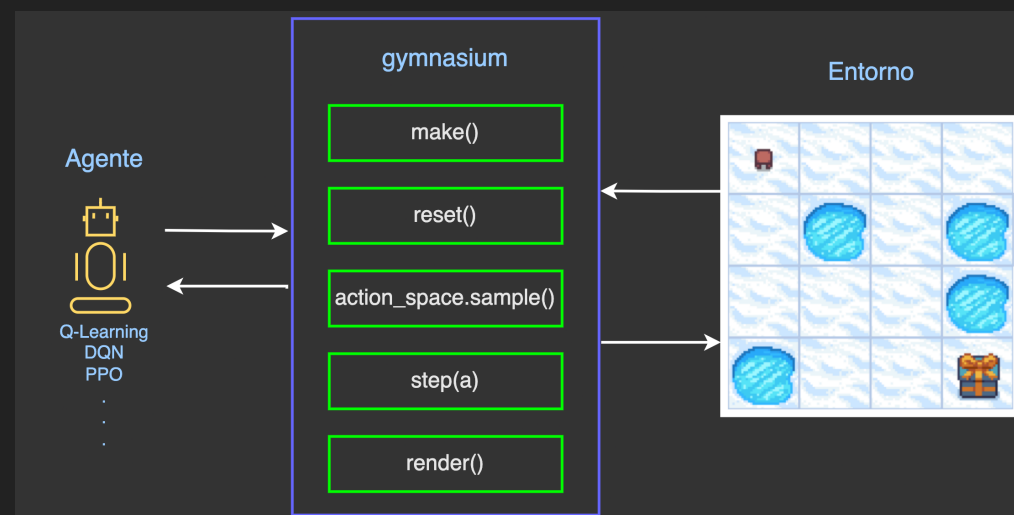
- ▶ Gymnasium se implementa mediante una API entre el agente y el entorno.



COMO USAR GYMNASIUM?



- ▶ `make()` crea el entorno (una instancia).
- ▶ `reset()` establece al estado inicial al agente en cada episodio y devuelve s_0 .
- ▶ `action_space.sample()` selecciona en forma aleatoria una acción posible desde el estado que el agente observa.
- ▶ `step(a)` ejecuta la acción que seleccionó y devuelve el nuevo estado y la recompensa.
- ▶ `render()` grafica el entorno.



COMO USAR GYMNASIUM?



```
import gymnasium as gym
```

```
import matplotlib.pyplot as plt
```

```
env = gym.make("FrozenLake-v1", render_mode="rgb_array") # Crear entorno "FrozenLake-v1" o "ALE/Breakout-v5"
```

```
obs, info = env.reset() # Reiniciar el entorno. En info puede haber ejemplo de {'lives': 5}
```

```
done = False
```

```
final_frame = None # Almacenar el último frame para renderizar al final
```

```
while not done:
```

```
    action = env.action_space.sample() # Seleccionar acción aleatoria
```

```
    obs, reward, done, truncated, info = env.step(action) # Ejecutar la acción
```

```
    final_frame = env.render() # Renderizar el entorno y almacenar el frame actual
```

```
plt.imshow(final_frame) # Mostrar el último frame al final del episodio
```

```
plt.axis("off")
```

```
plt.show()
```

```
env.close() # Cerrar el entorno al finalizar
```



COMO USAR GYMNASIUM?



```
import gymnasium as gym
```

```
import matplotlib.pyplot as plt
```

```
env = gym.make("FrozenLake-v1", render_mode="rgb_array") # Cargar el entorno "FrozenLake-v1"
```

```
obs, info = env.reset() # Reiniciar el entorno
```

```
done = False
```

```
final_frame = None # Almacenar el último frame para renderizar al final
```

```
while not done:
```

```
    action = env.action_space.sample() # Seleccionar acción aleatoria
```

```
    obs, reward, done, truncated, info = env.step(action) # Ejecutar la acción
```

```
    final_frame = env.render() # Renderizar el entorno y almacenar el frame actual
```

```
    print(f"Estado: {obs}, Recompensa: {reward}, Finalizado: {done}, Truncado: {truncated}")
```

```
plt.imshow(final_frame) # Mostrar el último frame al final del episodio
```

```
plt.axis("off")
```

```
plt.show()
```

```
env.close() # Cerrar el entorno al finalizar
```



EJEMPLO



```
Estado: 0, Recompensa: 0.0, Finalizado: False, Truncado: False  
Estado: 1, Recompensa: 0.0, Finalizado: False, Truncado: False  
Estado: 5, Recompensa: 0.0, Finalizado: True, Truncado: False
```



COMO USAR GYMNASIUM?



```
import gymnasium as gym
```

```
import matplotlib.pyplot as plt
```

```
env = gym.make("FrozenLake-v1", render_mode="rgb_array") # Crear entorno "FrozenLake-v1" o "ALE/Breakout-v5"
```

```
print(env.spec.max_episode_steps) # Muestra 100
```

```
obs, info = env.reset() # Reiniciar el entorno. En info puede haber ejemplo de {'lives': 5}
```

```
done = False
```

```
final_frame = None # Almacenar el último frame para renderizar al final
```

```
while not done:
```

```
    action = env.action_space.sample() # Seleccionar acción aleatoria
```

```
    obs, reward, done, truncated, info = env.step(action) # Ejecutar la acción
```

```
    final_frame = env.render() # Renderizar el entorno y almacenar el frame actual
```

```
plt.imshow(final_frame) # Mostrar el último frame al final del episodio
```

```
plt.axis("off")
```

```
plt.show()
```

```
env.close() # Cerrar el entorno al finalizar
```



COMO USAR GYMNASIUM?



```
import gymnasium as gym
```

```
import matplotlib.pyplot as plt
```

```
env = gym.make("FrozenLake-v1", render_mode="rgb_array") # Crear entorno "FrozenLake-v1" o "ALE/Breakout-v5"
```

```
print(env.spec.max_episode_steps) # Muestra 100
```

```
env = gym.wrappers.TimeLimit(env, max_episode_steps=50)
```

```
obs, info = env.reset() # Reiniciar el entorno. En info puede haber ejemplo de {'lives': 5}
```

```
done = False
```

```
final_frame = None # Almacenar el último frame para renderizar al final
```

```
while not done:
```

```
    action = env.action_space.sample() # Seleccionar acción aleatoria
```

```
    obs, reward, done, truncated, info = env.step(action) # Ejecutar la acción
```

```
    final_frame = env.render() # Renderizar el entorno y almacenar el frame actual
```

```
plt.imshow(final_frame) # Mostrar el último frame al final del episodio
```

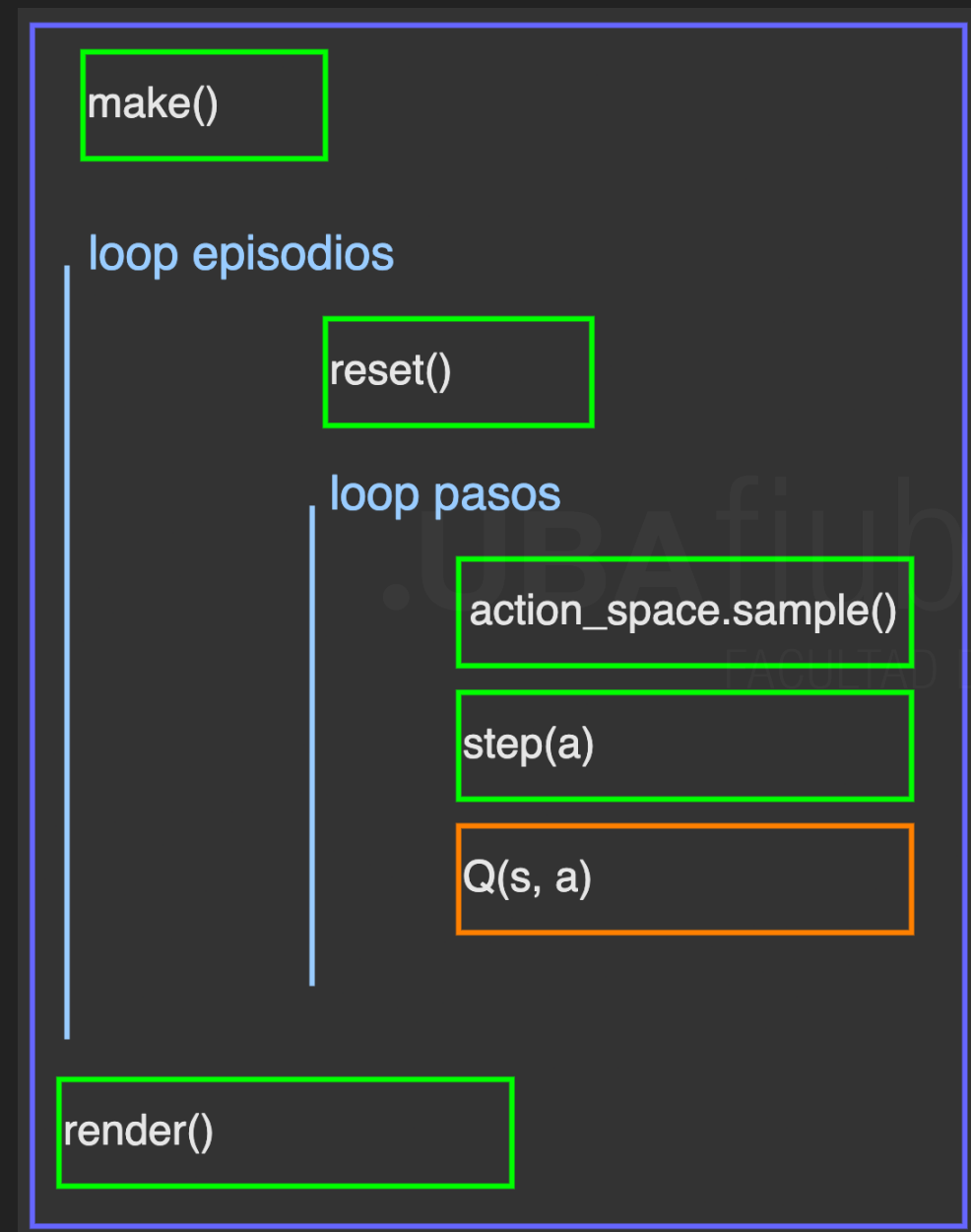
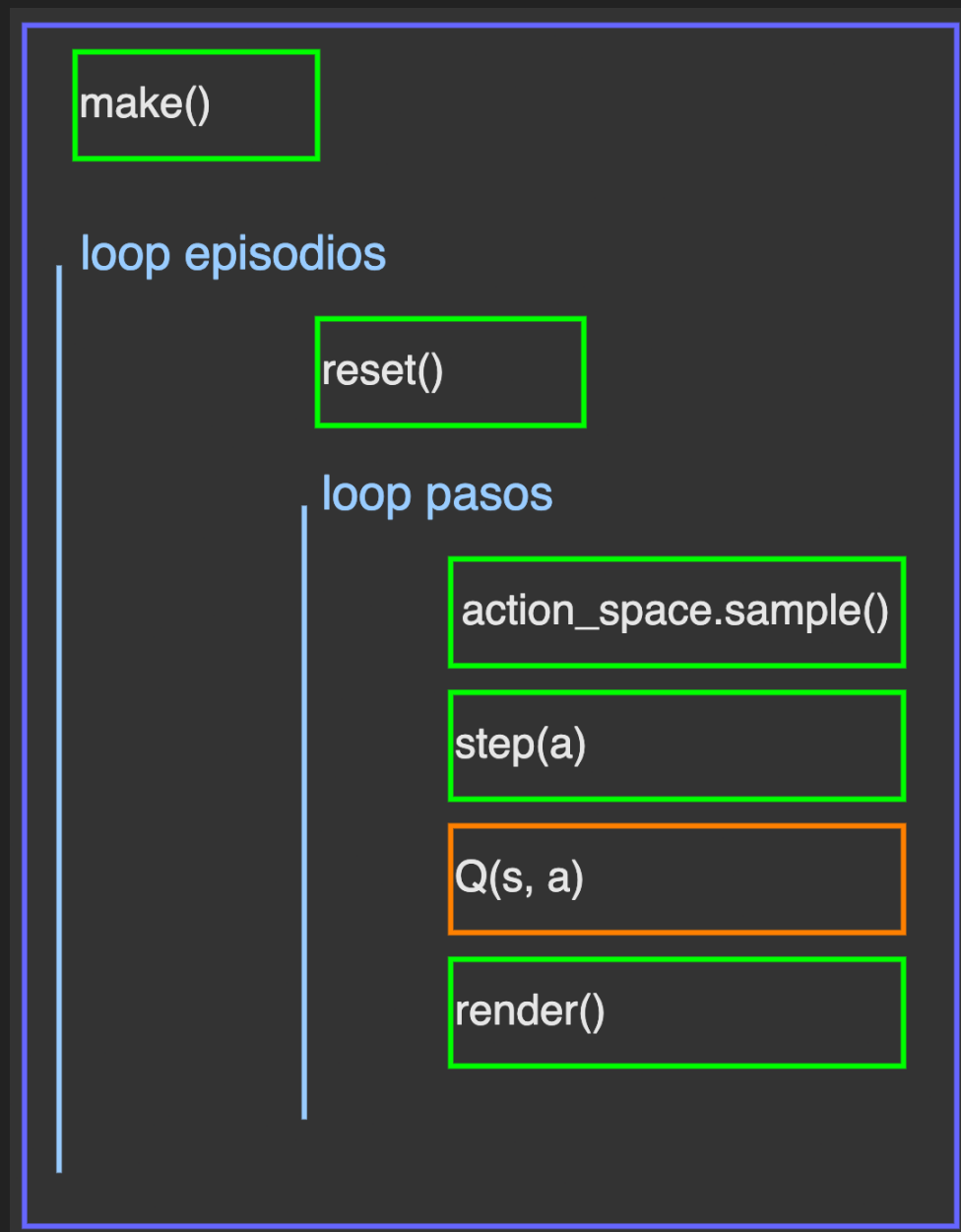
```
plt.axis("off")
```

```
plt.show()
```

```
env.close() # Cerrar el entorno al finalizar
```



IMPLEMENTACIÓN CON Q-LEARNING



https://github.com/ae-ar-uba/ar1/tree/main/src/scripts/q_learning_frozenlake

¿SE PUEDEN MODIFICAR LAS RECOMPENSAS?

- ▶ Si. Existen 2 formas:
 - ✓ **Directa** (externa)
 - ✓ **Wrapper**

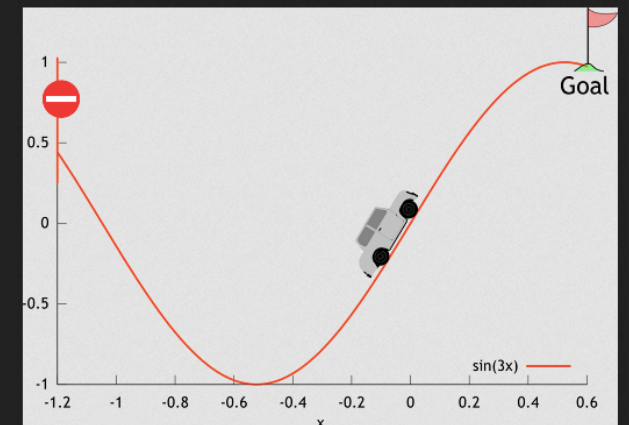
¿SE PUEDEN MODIFICAR LAS RECOMPENSAS?

► Directa:

- No se se puede cambiar la recompensa interna del entorno (está hardcodeada en Gymnasium).
- Pero, si es posible redefinir la recompensa después de recibirla en el bucle de entrenamiento. Por ejemplo:

```
nuevo_estado, recompensa, done, _, _ = env.step(accion)
# Sobrescribir la recompensa:
if nuevo_estado[0] >= 0.5: # Si alcanza la meta
    recompensa = 100 # Premio grande
elif accion == 2 and nuevo_estado[1] > 0: # Reforzar acelerar hacia arriba
    recompensa = 0.1 # Pequeño incentivo
```

- Esto no afecta al entorno, pero sí a cómo el agente aprende de sus acciones.
- La recompensa original es fija, pero se puede transformarla para mejorar el aprendizaje.



¿SE PUEDEN MODIFICAR LAS RECOMPENSAS?

► Wrapper:

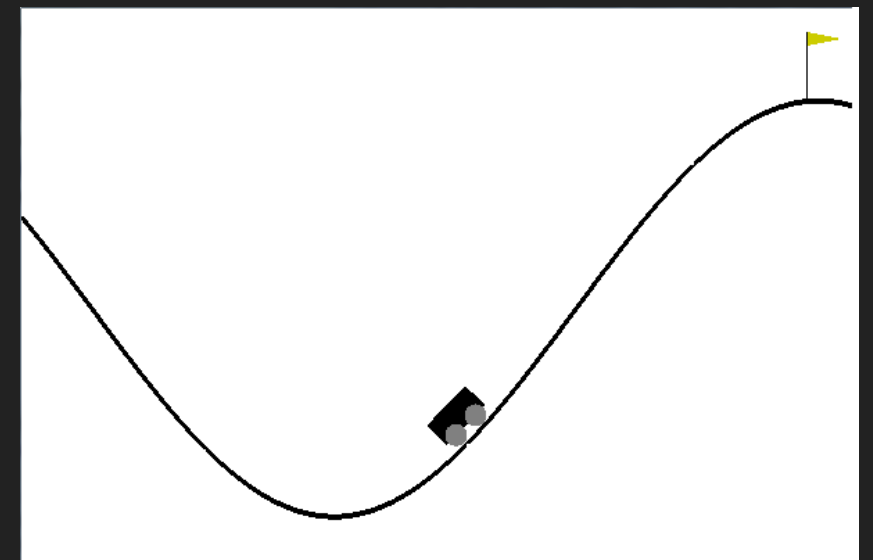
- Consiste en crear una subclase que modifique las recompensas de manera estructurada:

- ✓ esto hace el código más mantenible y escalable.
- ✓ es la forma más limpia y profesional de manejar recompensas customizadas.

```
from gymnasium.wrappers import RewardWrapper

class CustomRewardWrapper(RewardWrapper):
    def step(self, action):
        observation, reward, terminated, truncated, info = self.env.step(action)
        if observation[0] >= 0.5:
            reward = 100 # Recompensa por éxito
        elif action == 2 and observation[1] > 0: # Acelerar cuando sube
            reward = 0.1
        return observation, reward, terminated, truncated, info

# Usarlo al crear el entorno:
env = gym.make("MountainCar-v0")
env = CustomRewardWrapper(env)
```



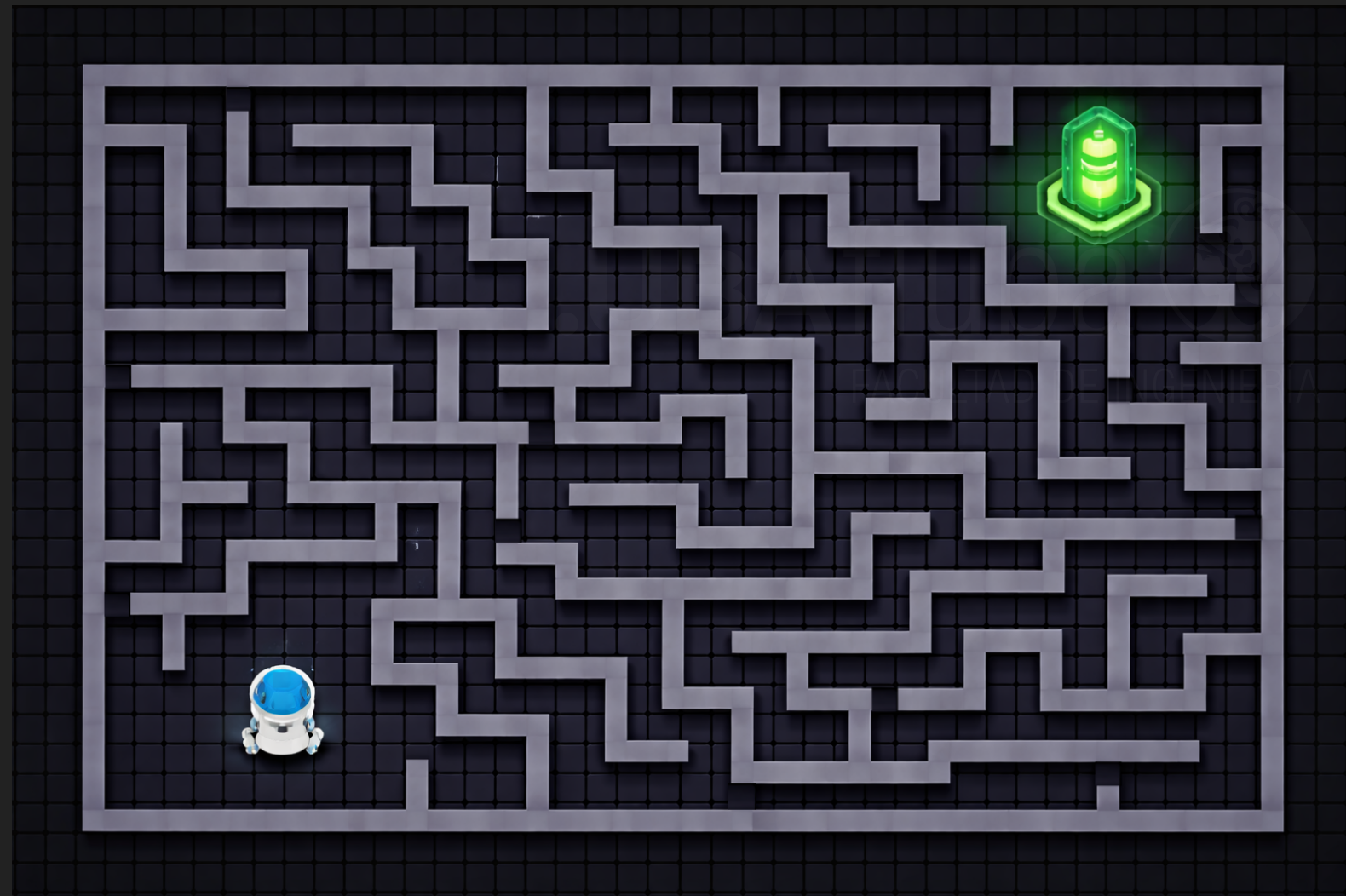
¿SE PUEDEN MODIFICAR LAS RECOMPENSAS?

- ▶ Un wrapper es una clase que envuelve un entorno de Gymnasium para modificarlo sin alterar su código original.
- ▶ Funciona como una "capa intermedia" que intercepta las interacciones con el entorno (como `step()`, `reset()`) y permite:
 - ✓ Modificar recompensas.
 - ✓ Cambiar observaciones (estados).
 - ✓ Añadir condiciones de terminación personalizadas.
 - ✓ Limitar acciones o estados.
- ▶ Es una técnica para adaptar entornos estándar a necesidades específicas, manteniendo la compatibilidad con la API de Gymnasium.

REWARD SHAPING (MODELADO DE RECOMPENSAS)

► Reward:

- ✓ +1 al llegar
- ✓ 0 por dar un paso



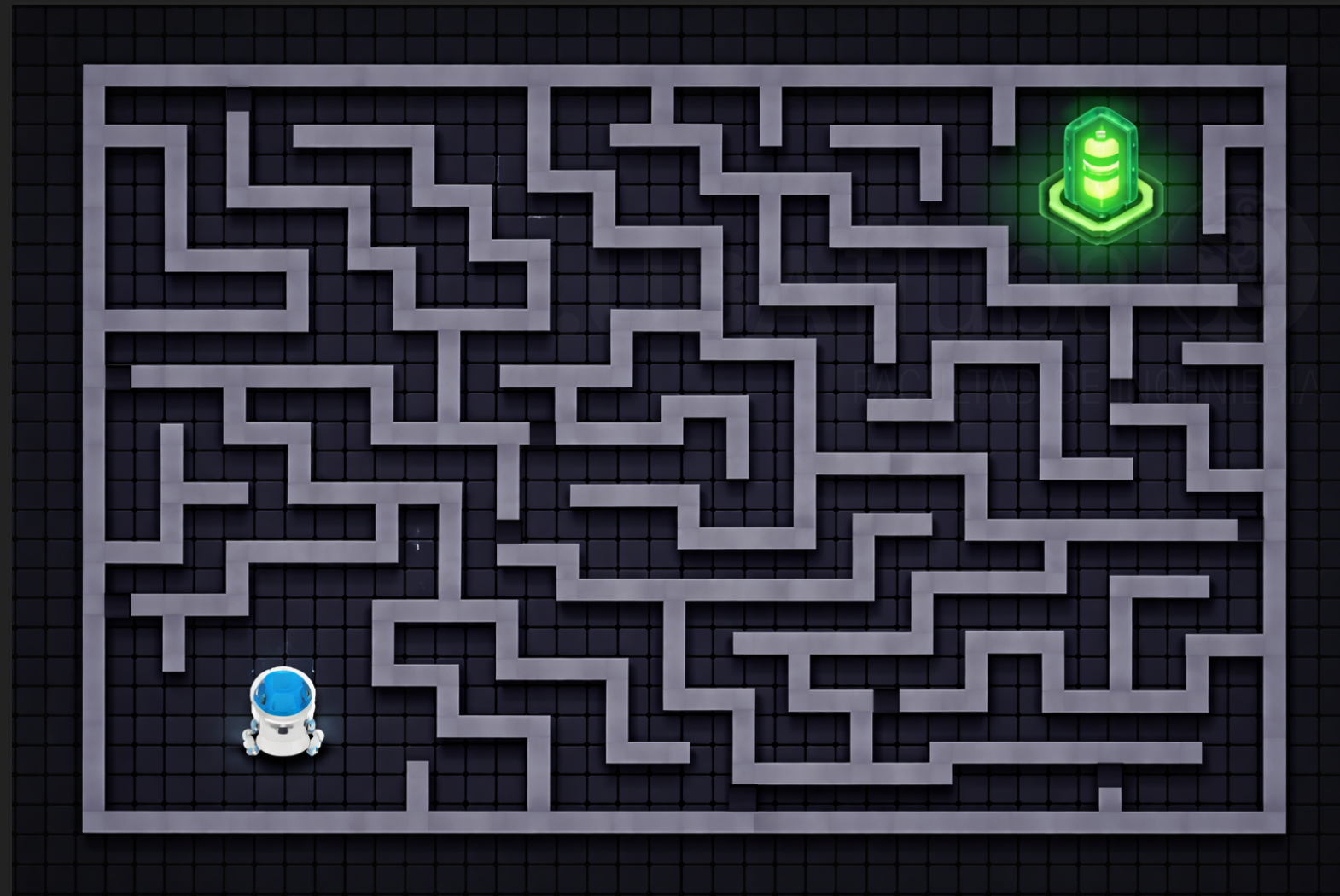
REWARD SHAPING (MODELADO DE RECOMPENSAS)

► Reward:

✓ +1 al llegar

✓ 0 por dar un paso

Reward Sparsity



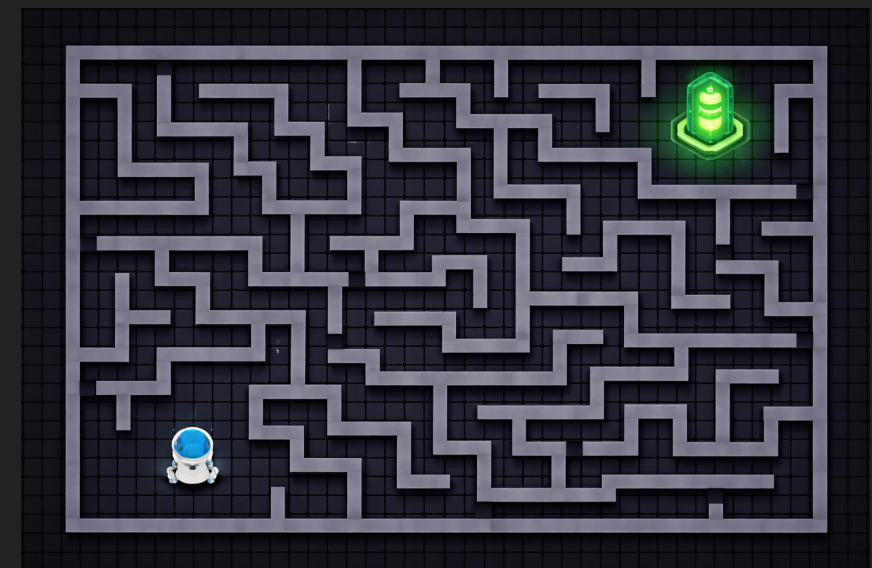
REWARD SHAPING (MODELADO DE RECOMPENSAS)

► Cuando existe **Reward Sparsity**:

- ✓ El agente se mueve al azar durante muchísimo tiempo
- ✓ La probabilidad de encontrar la salida es bajísima
- ✓ No hay señal de aprendizaje intermedia

► Resultado:

- ✓ Aprendizaje lento o nulo
- ✓ Alta varianza
- ✓ Puede no converger nunca en práctica



REWARD SHAPING (MODELADO DE RECOMPENSAS)

► **Solución:** otorgar mini recompensas intermedias para guiar la exploración (**reward shaping**).

► **1. Reward por acercarse**

✓ +0.01 si reduce la distancia a la estación

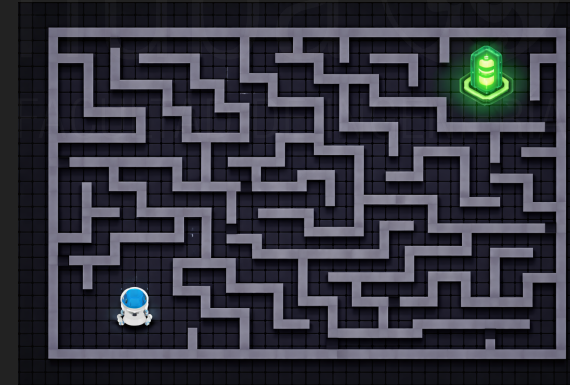
✓ -0.01 si se aleja

► **2. Penalización por tiempo**

✓ -0.001 por cada paso (evita que el robot deambule sin propósito)

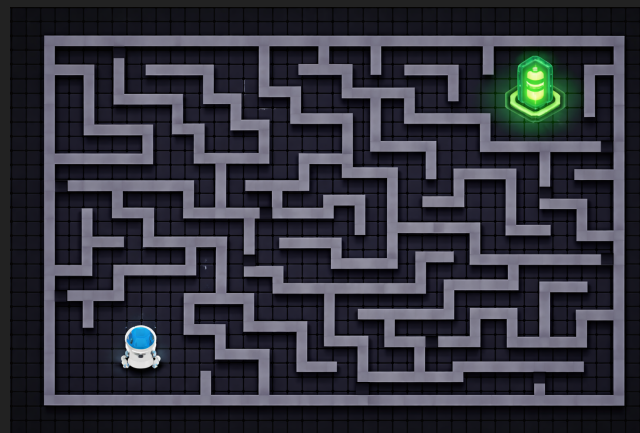
► **3. Reward por descubrir zonas nuevas**

✓ +0.05 si entra en una celda no visitada (fomenta exploración inteligente)

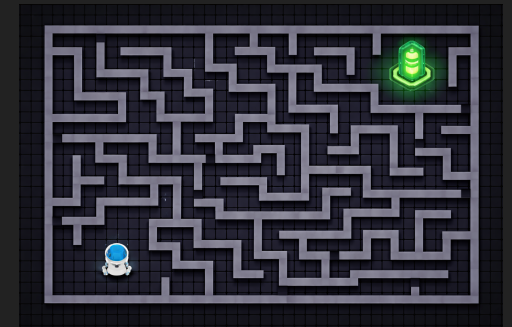


PROBLEMAS CON REWARD SHAPING

- ▶ Si se modela mal el Reward Shaping, el agente aprende otra cosa.
 - ✓ Si se premia demasiado explorar = **nunca termina el laberinto.**
 - ✓ Si se premia acercarse pero hay obstáculos = **puede quedar atrapado localmente.**
 - ✓ Si se penaliza mucho el tiempo = **toma rutas subóptimas.**



POTENTIAL-BASED REWARD SHAPING (PBRs)



- ▶ La recompensa basada en potencial se calcula como:

$$R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s)$$

- ▶ Donde:

- ▶ $\Phi(s)$: se denomina **función potencial** y mide qué tan cerca está el agente del objetivo (distancia euclidiana, distancia Manhattan, etc).

$$\Phi(s) = -\text{distancia_manhattan}(s, \text{goal})$$

POTENTIAL-BASED REWARD SHAPING (PBRs)

► Ejemplo:

✓ $\gamma = 0.9$

✓ Estado actual: distancia = 5

✓ Nuevo estado: distancia = 4 (se acercó)

► Entonces:

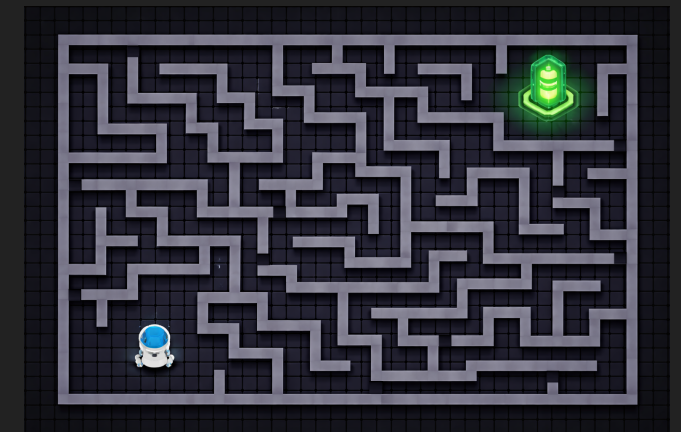
✓ $\Phi(s) = -5$

✓ $\Phi(s') = -4$

► Calculamos el shaping:

$$\gamma\Phi(s') - \Phi(s) = 0.9(-4) - (-5) = -3.6 + 5 = 1.4$$

► Resultado: **+1.4** de reward extra (se acercó = recompensa positiva)



POTENTIAL-BASED REWARD SHAPING (PBRs)

► En Q-Learning:

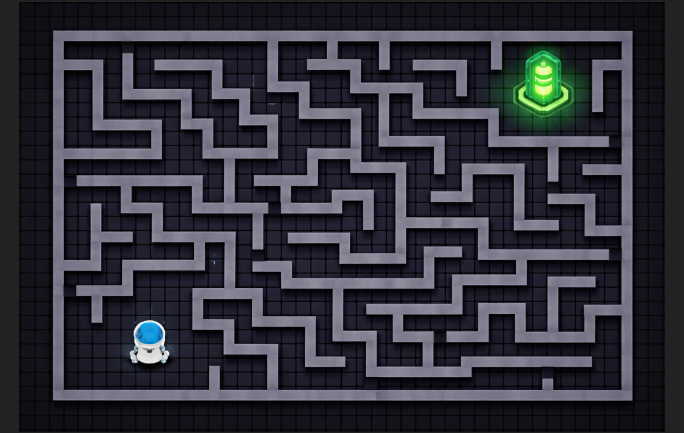
✓ Sin shaping:

$$Q[s,a] = Q[s,a] + \alpha * (r + \gamma * \max(Q[s']) - Q[s,a])$$

✓ Con shaping:

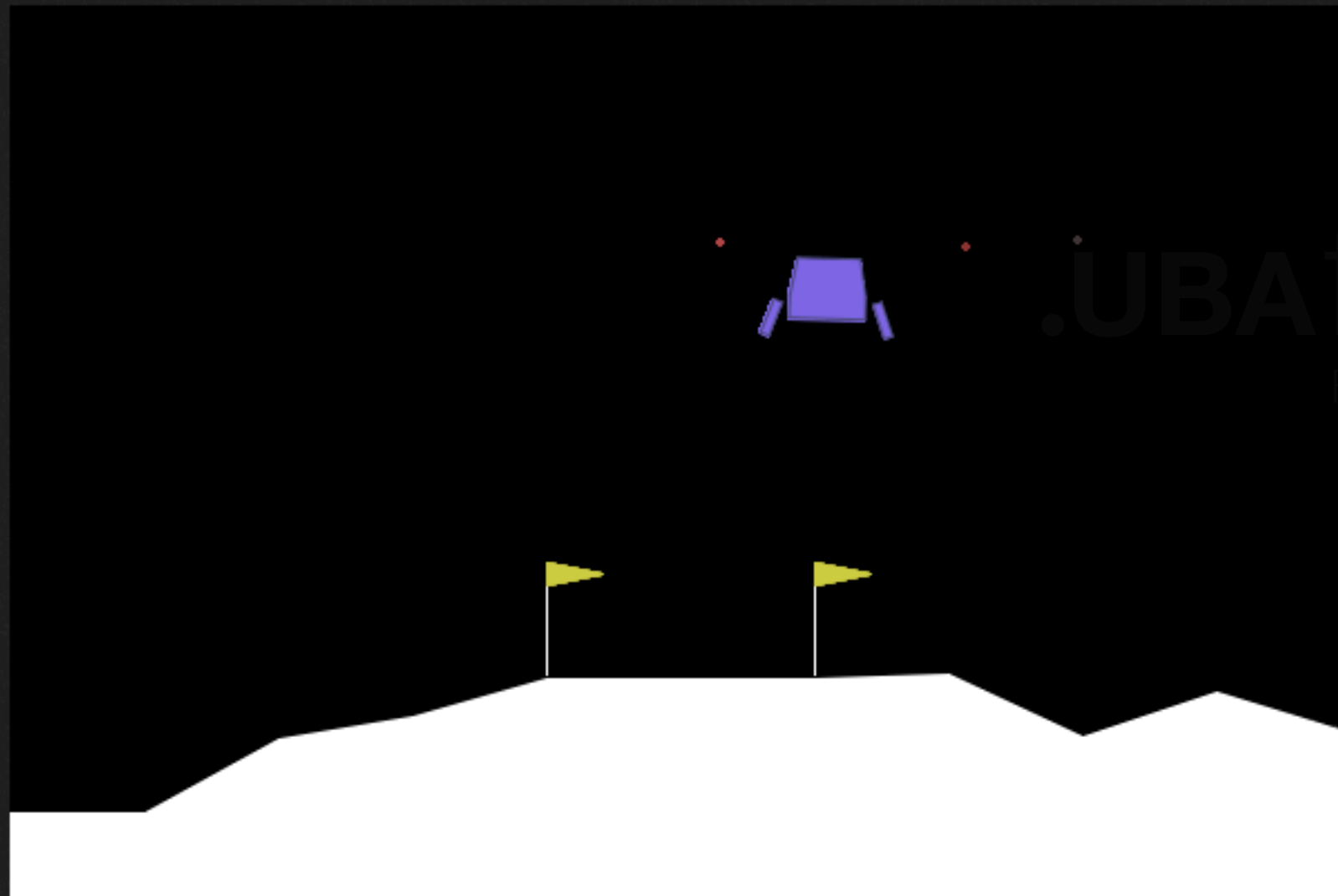
$$r_{\text{shaped}} = r + \gamma * \Phi(s') - \Phi(s)$$

$$Q[s,a] = Q[s,a] + \alpha * (r_{\text{shaped}} + \gamma * \max(Q[s']) - Q[s,a])$$



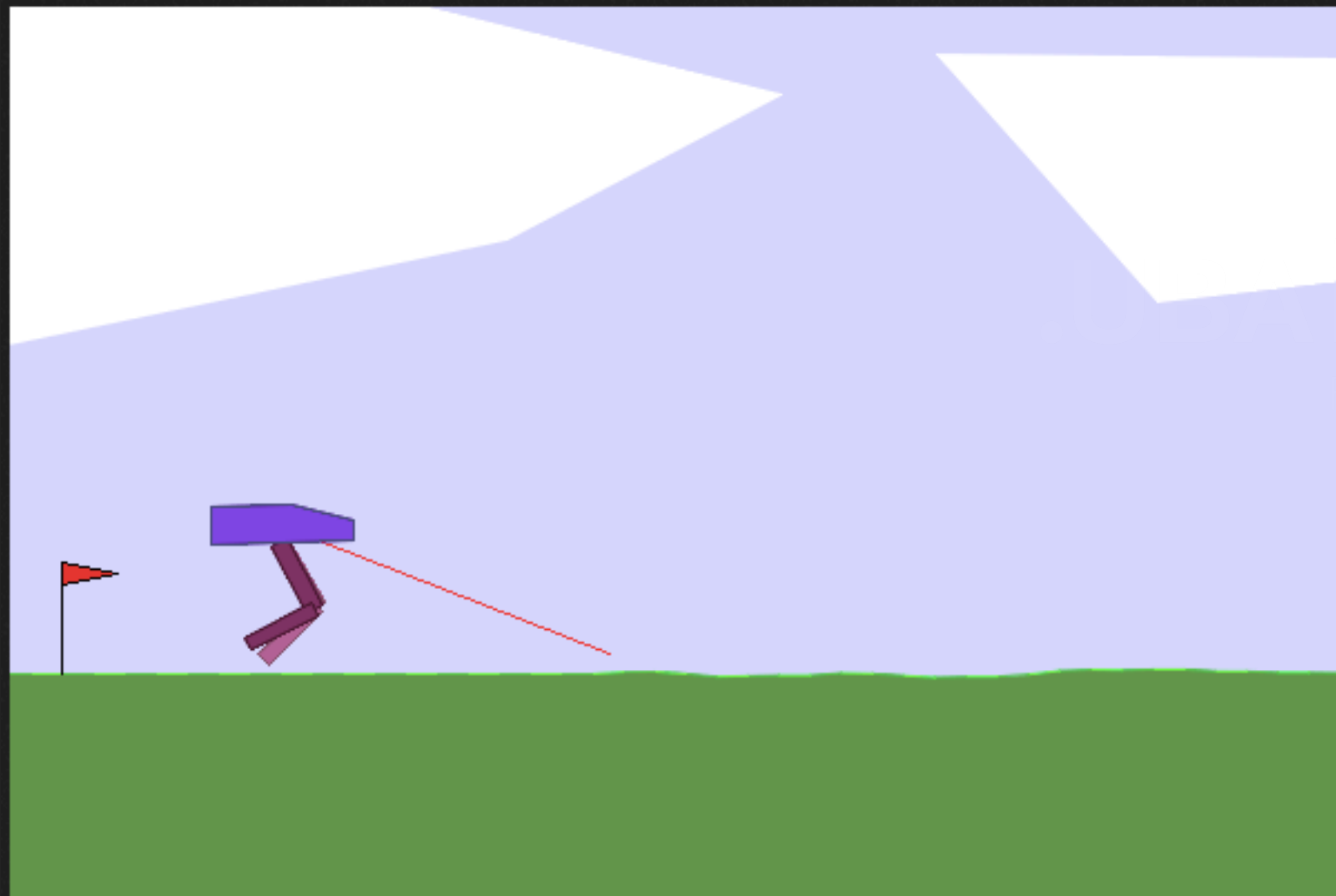
ENTORNOS CLÁSICOS DE GYMNASIUM

► LunarLander-v2



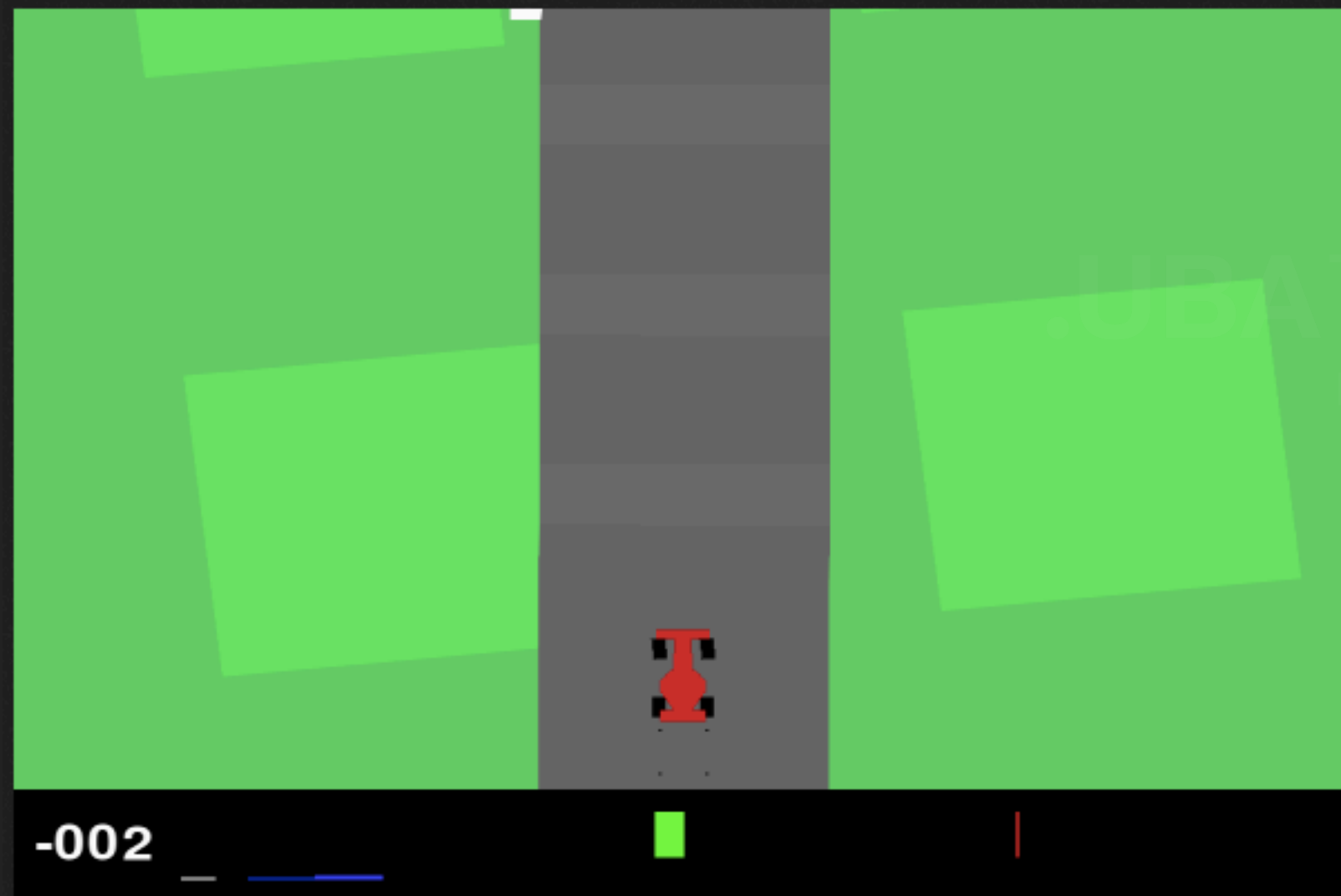
ENTORNOS CLÁSICOS DE GYMNASIUM

► BipedalWalker-v3



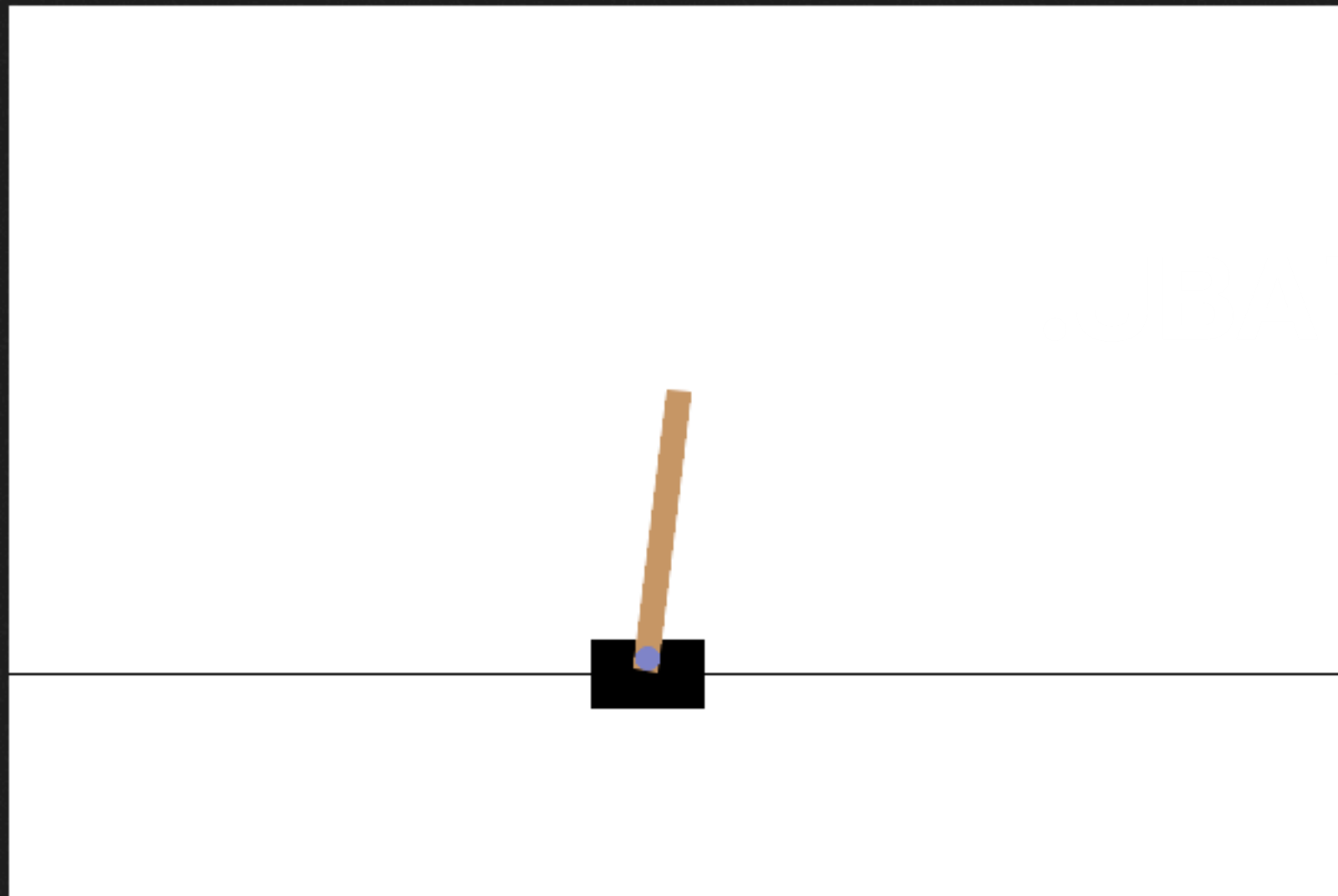
ENTORNOS CLÁSICOS DE GYMNASIUM

► CarRacing-v2



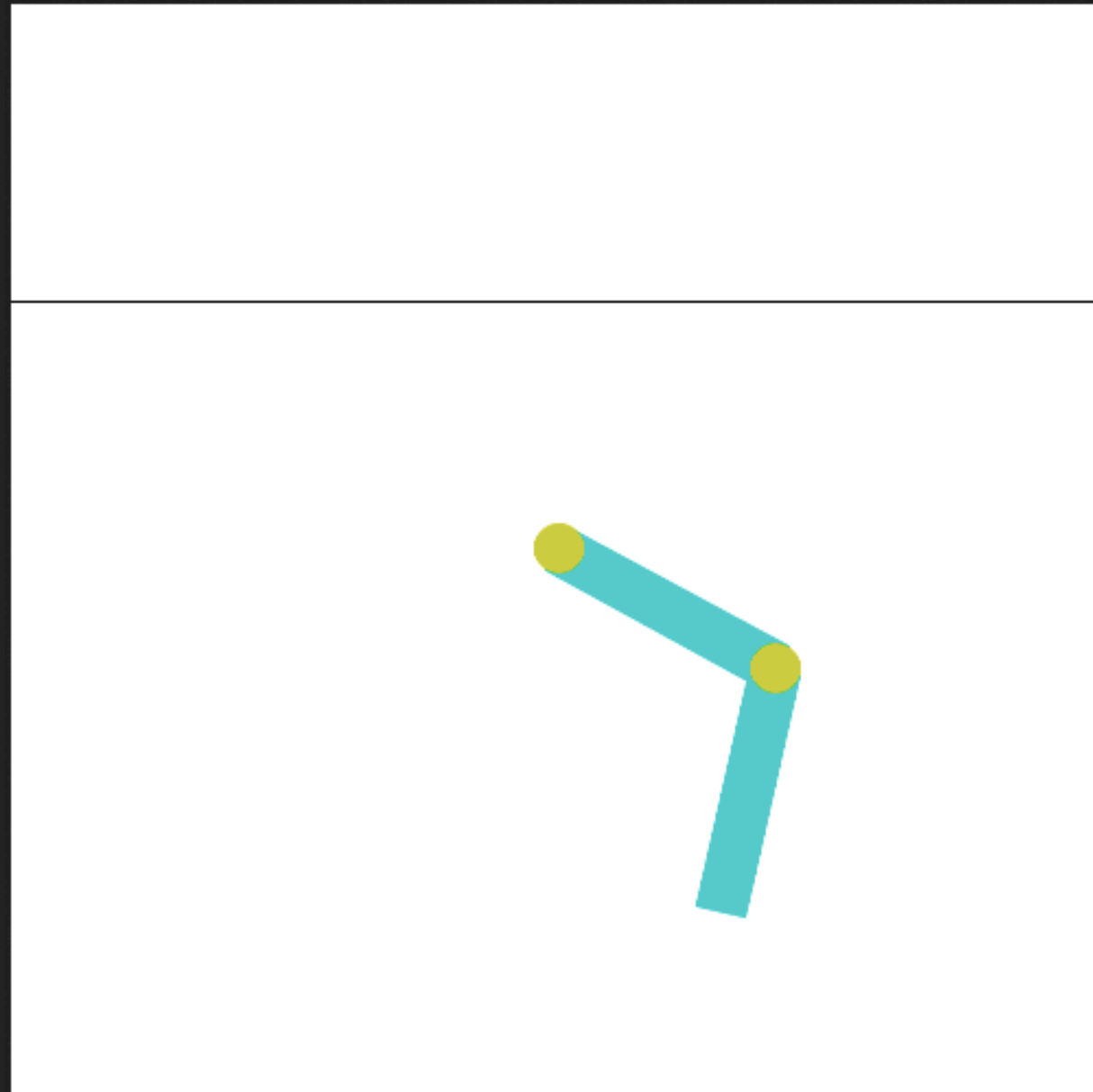
ENTORNOS CLÁSICOS DE GYMNASIUM

► CartPole-v1



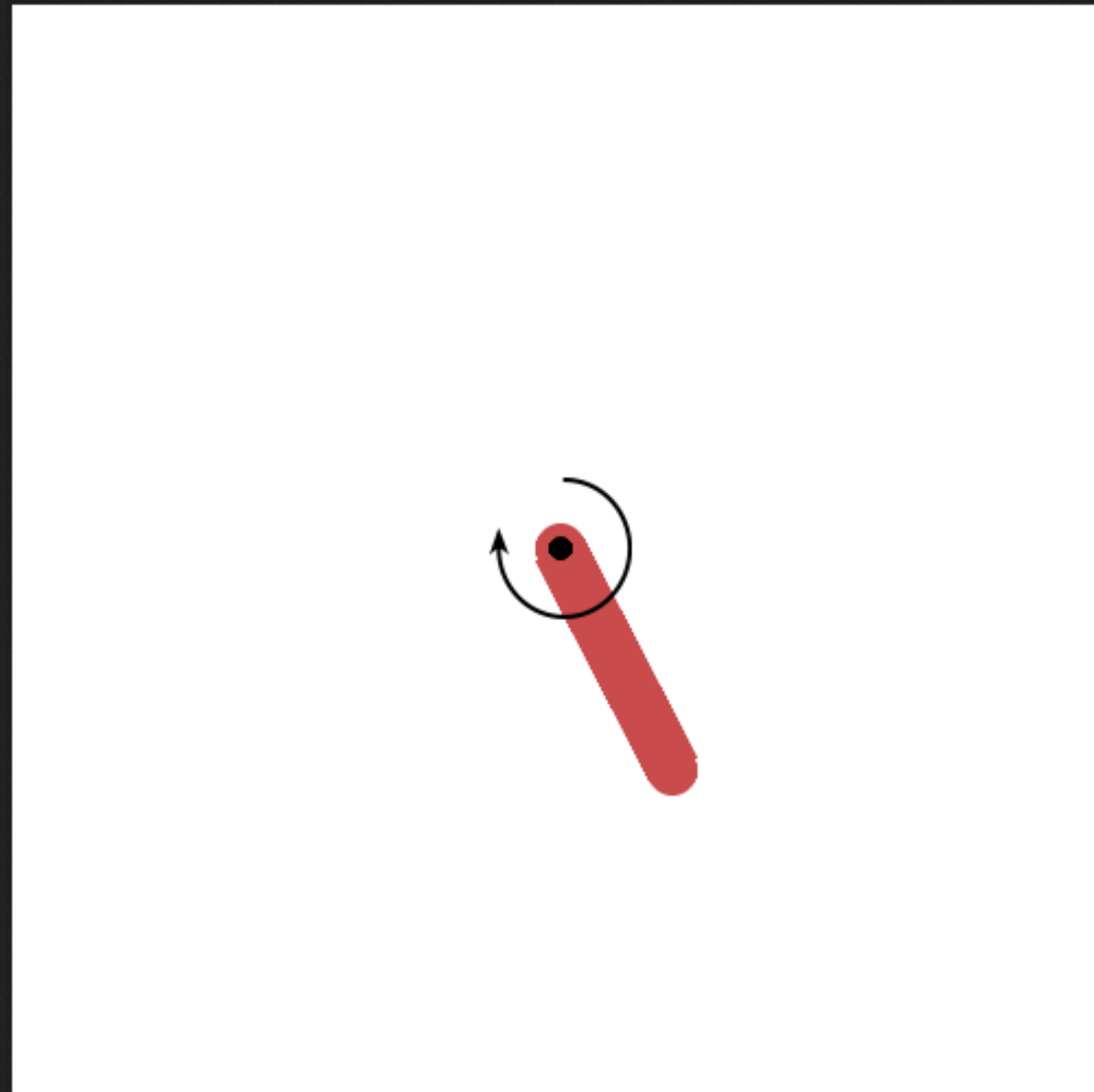
ENTORNOS CLÁSICOS DE GYMNASIUM

► Acrobot-v1



ENTORNOS CLÁSICOS DE GYMNASIUM

► Pendulum-v1



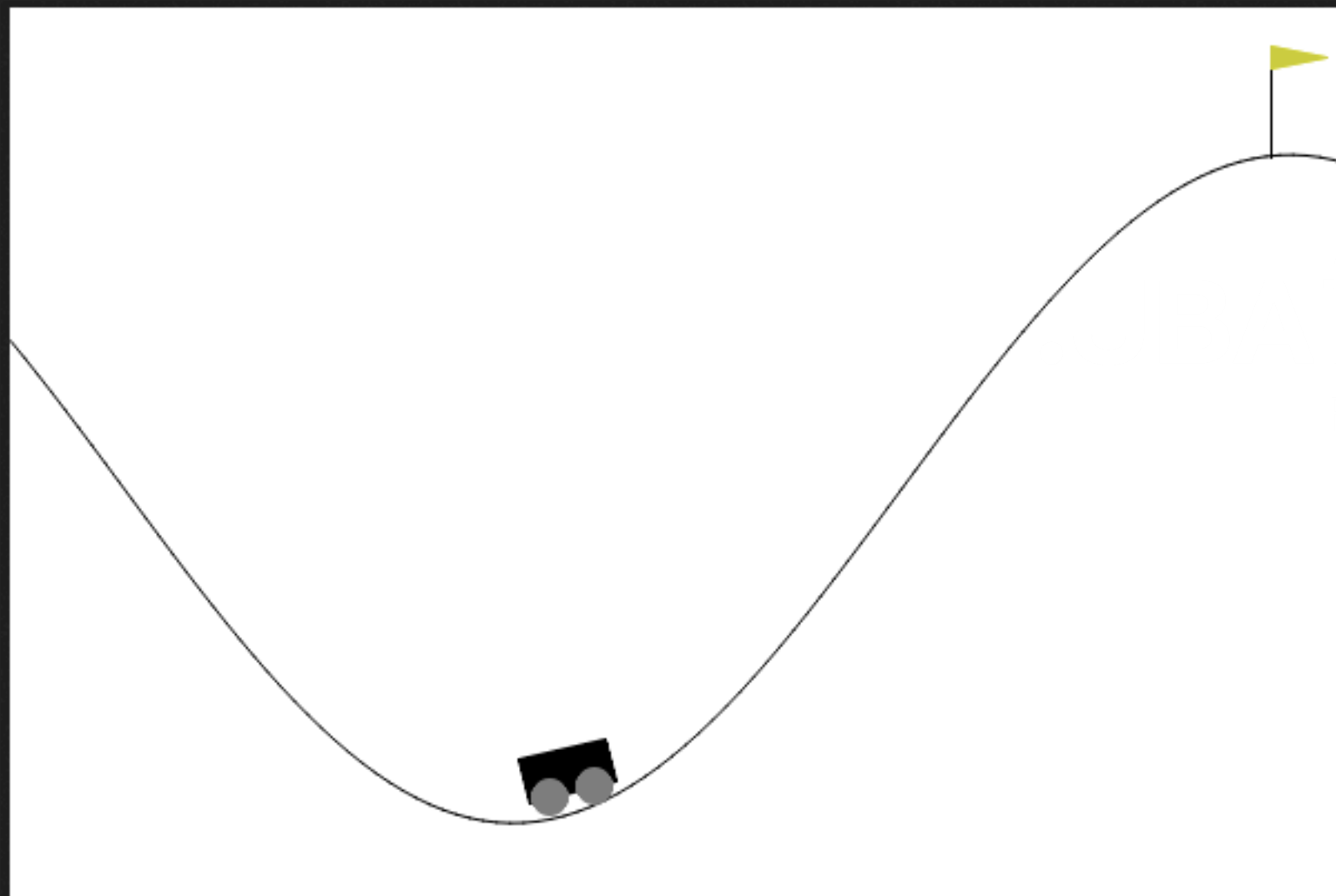
ENTORNOS CLÁSICOS DE GYMNASIUM

► FrozenLake-v1



ENTORNOS CLÁSICOS DE GYMNASIUM

► MountainCar-v0



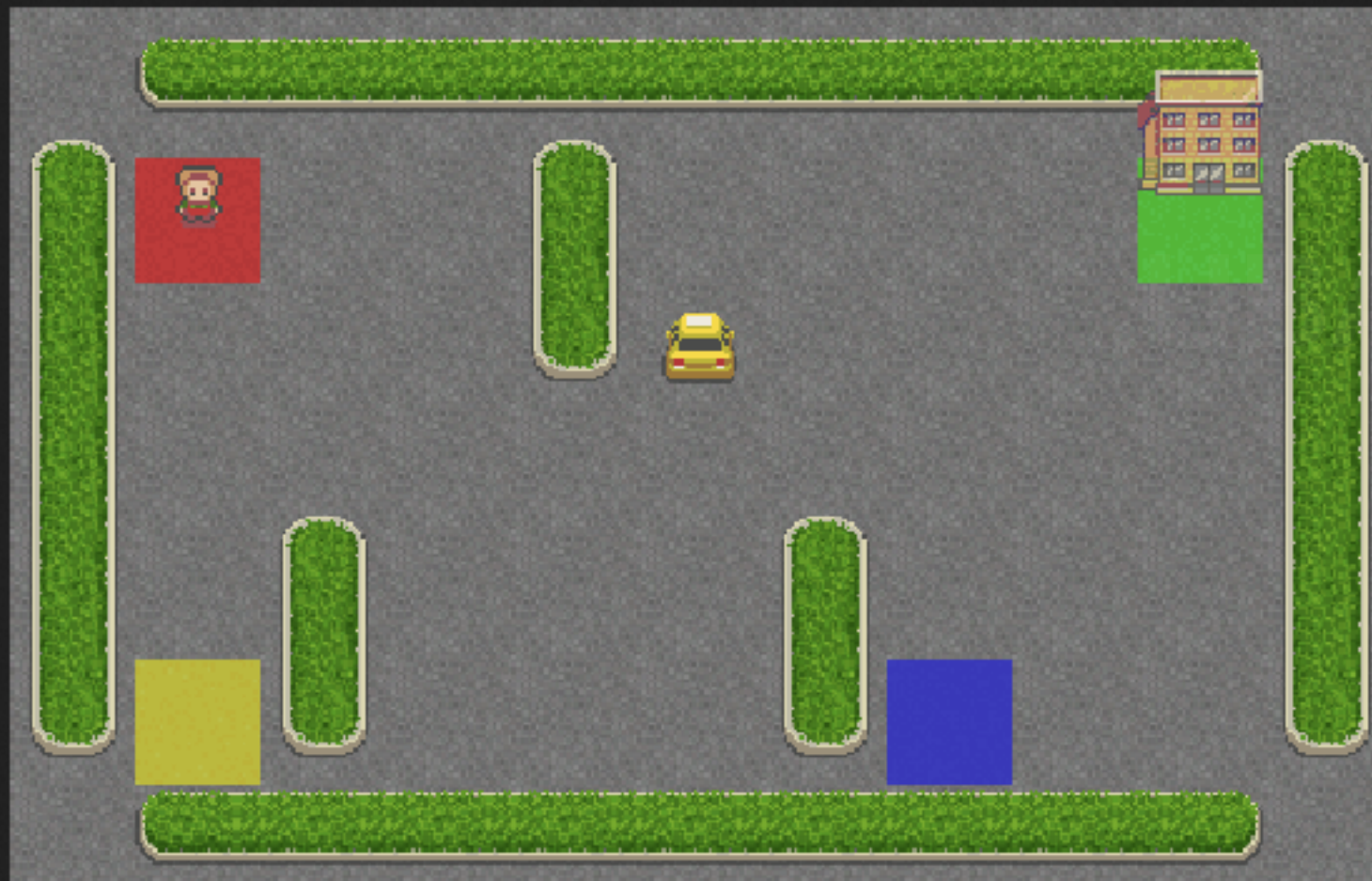
ENTORNOS CLÁSICOS DE GYMNASIUM

► Blackack-v1



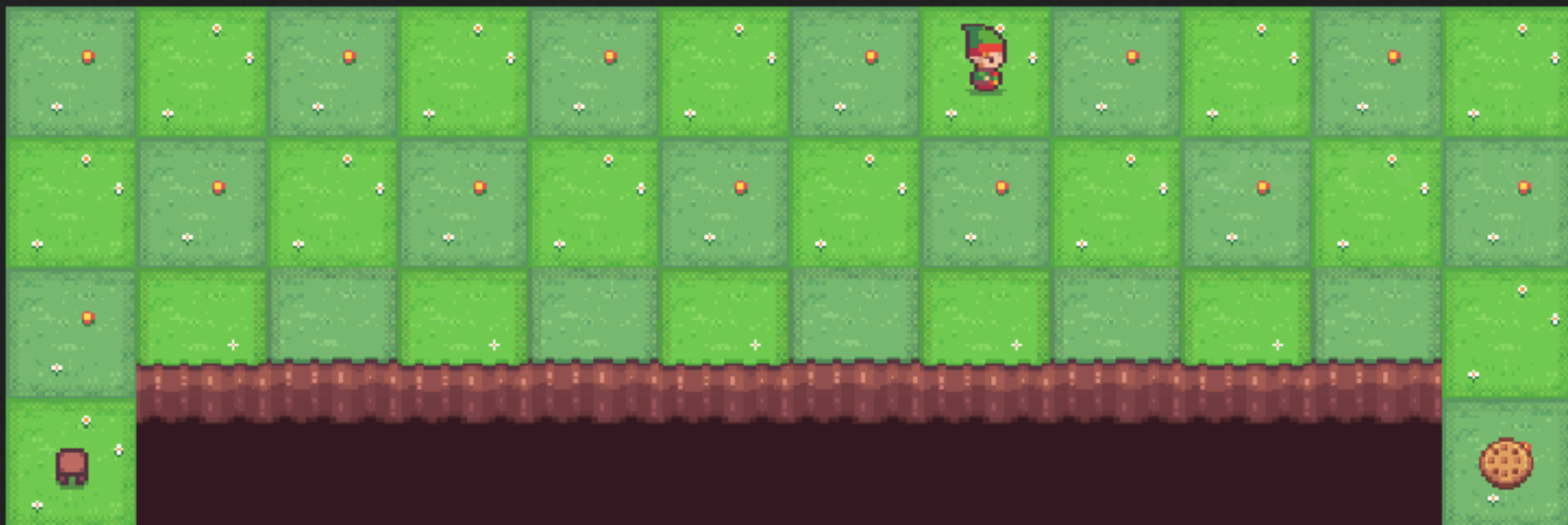
ENTORNOS CLÁSICOS DE GYMNASIUM

► Taxi-v3



ENTORNOS CLÁSICOS DE GYMNASIUM

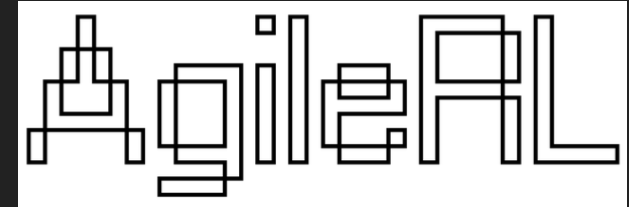
► CliffWalking-v0



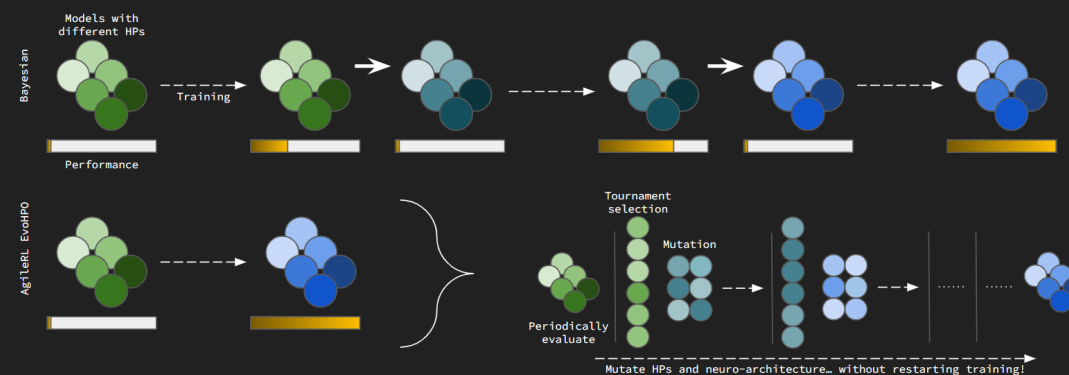
AGENTES



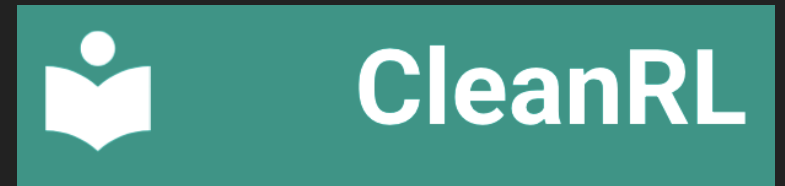
AGILERL



- ▶ **Propósito:** Entrenamiento de algoritmos de RL con aprendizaje activo y evolución adaptativa.
- ▶ **Uso principal:** AutoRL (optimización automática de agentes).
- ▶ **Características clave:**
 - ✓ Selección automática de modelos y arquitecturas.
 - ✓ Evaluación continua del desempeño.
 - ✓ Bueno para experimentación con diferentes estrategias.
- ▶ Optimización de hiperparámetros (**HPO**) evolutiva (basada en algoritmos genéticos) que reduce los tiempos de entrenamiento..
- ▶ **Agentes que provee:** DQN, PPO, A2C, SAC, TD3, DDPG, MADDPG, MATD3, IPPO.



CLEANRL

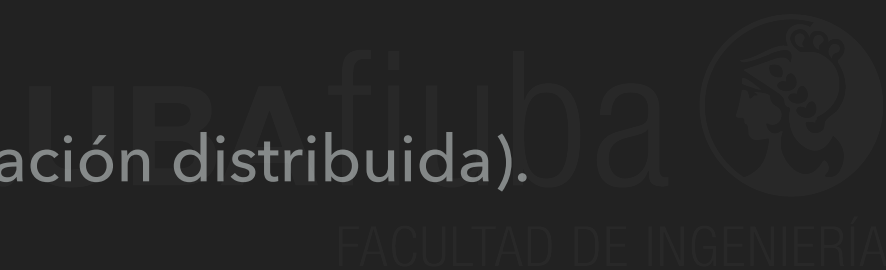


- ▶ **Propósito:** Proveer implementaciones limpias y mínimas de algoritmos de RL.
- ▶ **Uso principal:** Estudio, enseñanza y depuración de algoritmos de RL.
- ▶ **Características clave:**
 - ✓ Código corto y claro, ideal para aprender.
 - ✓ Basado en PyTorch.
 - ✓ Cada script suele ser autocontenido (1 archivo .py por algoritmo).
- ▶ **Agentes que provee:** DQN, C51, PPO, SAC, TD3, DDPG, PPG.

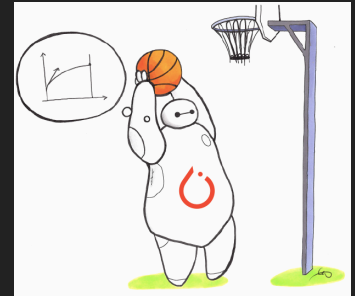
RL LIB



- ▶ **Propósito:** Entrenamiento escalable de RL distribuido y producción (cluster).
- ▶ **Uso principal:** Aplicaciones de RL a gran escala.
- ▶ **Características clave:**
 - ✓ Esta basada en Ray (el motor que permite computación distribuida).
 - ✓ Tolera entrenamiento paralelo y multinodo.
 - ✓ Ray incluye **Tune** (ajuste de hiperparámetros) y **Serve** (despliegue de modelos).
- ▶ **Agentes que provee:** DQN, A3C (deprecated), PPO, SAC, TD3, DDPG, IMPALA, APEX.
- ▶ Puede trabajar con múltiples agentes.

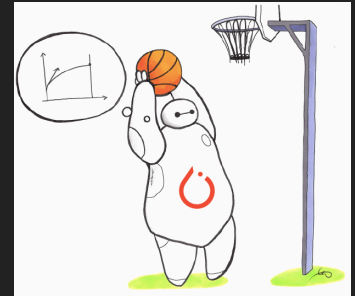


STABLE BASELINE 3



- ▶ **Propósito:** Implementaciones robustas y listas para usar de algoritmos de RL.
- ▶ **Uso principal:** Entrenamiento rápido y reproducible de agentes RL.
- ▶ **Características clave:**
 - ✓ Basado en PyTorch.
 - ✓ Fácil de usar con `model.learn()` y `model.predict()`.
 - ✓ Excelente para prototipado rápido y experimentos controlados.
- ▶ **Agentes que provee:** DQN, A2C, PPO, HER, SAC, TD3, DDPG.
- ▶ Soporta también acciones continuas y discretas y multiprocessing.

STABLE BASELINE 3 – EJEMPLO SIMPLE



```
import gymnasium as gym
```

```
from stable_baselines3 import DQN
```

```
env = gym.make("MountainCar-v0", render_mode=None) # Crea el entorno
```

```
model = DQN("MlpPolicy", env, verbose=1) # Crea el modelo DQN (0 silencio, 1 c/ ciertos pasos, 2 todo)
```

```
model.learn(total_timesteps=10_000) # Entrena el modelo por 10,000 pasos
```

```
# evaluacion del agente (sin entrenar más)
```

```
obs, _ = env.reset()
```

```
done = False
```

```
while not done:
```

```
    action, _states = model.predict(obs)
```

```
    obs, reward, done, truncated, info = env.step(action)
```

```
    env.render()
```

```
env.close()
```


REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- ▶ <https://stable-baselines3.readthedocs.io/en/master/>
- ▶ <https://docs.cleanrl.dev/>
- ▶ <https://docs.agilerl.com/en/latest/>
- ▶ <https://docs.ray.io/en/latest/rllib/index.html>
- ▶ <https://gymnasium.farama.org/>
- ▶ <https://github.com/Farama-Foundation/PettingZoo>
- ▶ <https://github.com/google-deepmind/mujoco>
- ▶ <https://minihack.readthedocs.io/en/latest/>
- ▶ "Comparison of reinforcement learning algorithms applied to Humanoid-v2 in MuJoCo using CleanRL" <https://www.youtube.com/watch?v=n0Cpqzqzroo>